

时序图未来链接预测

参赛团队：ZenFlow

本项目是第六届计图人工智能挑战赛“基于图学习的动态推荐任务”的完整复现代码。给定历史时序交互图，以及测试查询中的源节点、时间戳和 100 个候选目标节点，模型输出每个候选在未来形成链接的相对概率。

项目包含 A 榜 Dataset1/2 与 B 榜 Dataset3/4 建模流水线，从赛事原始 CSV 开始自动完成数据检查、验证集构造、特征生成、模型训练、推理、结果校验及 ZIP 打包。默认入口运行 B 榜。

1. 主要特点

- 使用 Jittor/JittorGeometric 训练时序图与图表示模型；
- 将未来链接预测建模为每行 100 个候选节点的学习排序问题；
- 融合 CRAFT、时序统计、局部图结构、矩阵分解、LightGCN、协同过滤和树模型；
- Dataset2 使用滚动时间窗口和 OOF 预测训练多级重排序器；
- 大型特征采用稀疏矩阵、mmap 和分块计算；
- Dataset4 使用紧凑二部图索引，未见候选走冷启动特征，不构造百万级稠密矩阵；
- NumPy/SciPy、LightGBM 及 Dataset3 分块特征支持受控多线程；
- 支持阶段缓存、断点续跑和指定阶段强制重跑；
- 自动检查预测矩阵形状、数值范围并生成 `result.zip`。

2. 目录结构

```
1  .
2  |— run.py                      # 完整流水线入口
3  |— requirements.txt           # Python 依赖
4  |— scripts/
5  |   |— run_pipeline.sh        # 从零训练、推理和打包入口
6  |   |— run_checkpoint_inference.py # 直接加载归档权重推理并打包
7  |— code/
8  |   |— solution/
9  |   |   |— base/              # Dataset2 基础模型与公共特征
10 |   |   |— dataset1/          # Dataset1 训练、特征和推理
11 |   |   |— dataset2/         # Dataset2 高层特征与重排序
12 |   |   |— dataset3/         # Dataset3 训练与推理
13 |   |   |— dataset4/         # Dataset4 图表示、时序和排序流水线
14 |   |— training/
15 |       |— base_models/       # Dataset2 基础模型训练
16 |       |— dataset2/          # OOF、校准及上层模型训练
17 |— README.md
```

提交审核包另外包含 `checkpoints/`（直接推理所需权重）、`scripts/`（统一运行与校验脚本）和根目录 `README.pdf`。

运行过程中生成的模型、缓存和预测位于：

```
1 | outputs/
2 | submissions/
```

这些目录已由 `.gitignore` 排除，可以从原始数据重新生成。

3. 环境要求

推荐环境：

```
1 | Ubuntu 22.04
2 | Python 3.10
3 | CUDA 12.4
4 | Jittor 1.3.10
5 | 18 GB 以上内存
6 | 具有充足空间的高速磁盘
```

Jittor 首次执行时需要编译算子，因此第一次启动通常明显慢于后续运行。

使用 venv 安装

Ubuntu 可使用系统提供的 `python3` 创建环境。系统依赖配置参考[Jittor 官方安装说明](#)，JittorGeometric 使用项目固定的[官方源码](#)：

```
1 | sudo apt-get update
2 | sudo apt-get install -y \
3 |     python3 python3-dev python3-venv python3-pip \
4 |     build-essential g++ libomp-dev git
5 |
6 | python3 -m venv .venv
7 | source .venv/bin/activate
8 | python3 -m pip install --upgrade pip setuptools wheel
9 | python3 -m pip install -r requirements.txt
```

其中 Jittor 由 PyPI 安装；JittorGeometric 使用 `requirements.txt` 中固定的 GitHub 提交源码包构建安装。后者包含标准 `setup.py`，因此不需要手动执行 `git clone` 和 `pip install .`。安装过程需要能够访问 GitHub。

`requirements.txt` 只能安装 Python 包，不能安装编译器、Python 开发头文件或 OpenMP 运行库，因此全新 Ubuntu 环境仍须先执行上述 `apt-get` 命令。

如 Python、编译器和 OpenMP 已由审核环境提供，可以跳过系统依赖安装。

使用 Conda（可选）

Conda 不是运行本项目的必要条件。如需使用 Conda，可以执行：

```
1 | conda create -n <env_name> -c conda-forge --override-channels \
2 |     'python=3.10.*' cpython pip -y
3 | conda activate <env_name>
4 | python3 -m pip install --upgrade pip setuptools wheel
5 | python3 -m pip install -r requirements.txt
```

检查环境

```
1 python3 --version
2 g++ --version
3 python3 -m jittor.test.test_example
4 python3 -c "from jittor_geometric.nn.models.craft import CRAFT; from
jittor_geometric.data_loader.temporal_data_loader import TemporalDataLoader,
get_neighbor_sampler; print('JittorGeometric CRAFT imports ok')"
5 python3 -c "import numpy, pandas, lightgbm, scipy; print('dependencies ok')"
```

如 CUDA 工具链不在标准位置，可以在运行前指定：

```
1 export cc_path=/usr/bin/g++
2 export nvcc_path=/usr/local/cuda/bin/nvcc
```

请根据审核机的实际 CUDA 安装路径调整 `nvcc_path`。CPU 环境可以完成 Jittor 基础安装测试，但本项目完整训练建议使用可用的 NVIDIA CUDA 环境。

4. 数据准备

将所需原始数据集放在项目根目录。B 榜目录为：

```
1 dataset3/
2 |— train.csv
3 |— test.csv
4
5 dataset4/
6 |— train.csv
7 |— test.csv
```

训练集字段：

```
1 src,dst,time
```

Dataset3/4 训练集还包含赛事提供的 `split` 字段。测试集字段为：

```
1 src,time,c1,...,c100
```

`run.py` 启动时会检查：

- 四个原始文件是否存在；
- CSV 表头是否符合预期；
- 测试文件是否包含 100 个候选列。

5. 运行方式

5.1 从零训练并生成 B 榜结果

完整运行：

```
1 | python run.py --target b --workers 8 --chunk-rows 2500 --cf-dim 128 --seed 20260724
```

单独运行某一数据集：

```
1 | python run.py --target dataset3 --workers 8 --seed 20260724
2 | python run.py --target dataset4 --workers 8 --chunk-rows 2500 --cf-dim 128 --seed 20260724
```

分别生成 `submissions/prediction/dataset3.csv` 和 `submissions/prediction/dataset4.csv`。两个文件完成后执行：

```
1 | python run.py --target package-b
```

最终生成包含两个 CSV 的 `submissions/prediction/result.zip`。

关键参数：

- `--workers`：CPU 并行线程数；18GB 内存建议为 8；
- `--chunk-rows`：Dataset4 推理分块行数，当前配置为 2500；
- `--cf-dim`：协同 CountSketch 维度，当前配置为 128；
- `--seed`：全流程随机种子，当前配置为 20260724；
- Dataset4 GraphMF：64 维、5 epoch、batch size 32768、5 个负样本、学习率 0.002；
- Dataset4 CRAFT：30 个时序邻居、20 epoch、batch size 256；
- Dataset4 最终排序器：候选级 LambdaRank。

训练保护与断点恢复

- Dataset3 LambdaRank 在按时间排序的最后 15% 查询上按榜单 MRR 早停，再用选中的轮数重训；
- Dataset3 LightGCN 每 200 step 保存嵌入快照，并根据训练损失平台停止；
- Dataset4 GraphMF 在 split0 尾部窗口按验证 BPR loss 选择 epoch，再分别用完整 split0 和完整 train 按最佳 epoch 重训；
- Dataset4 CRAFT 在 split0 内部尾窗按 AP 早停，再分别用完整 split0 和完整 train 按最佳 epoch 重训；每 epoch、最佳权重和最新权重均保存；
- Dataset4 Tree/Temporal/Graph/LightGCN 基础模型及六个分布/协同/Anchor/Group/Temporal-stack 上下文排序器沿用原有 NDCG/AP 时间后段早停；V4 仅将最终 LambdaRank 改为与榜单一致的 MRR 早停；四段式前向堆叠使基础模型、上下文模型、元模型和最终验证的标签区间按时间互不重合；
- Dataset4 CSV 预处理、验证特征、GraphMF、CRAFT、协同草图、LightGCN 层传播和正式推理均有阶段内进度点；

- 缓存签名包含数据文件、关键超参数和分层算法版本；上层堆叠变化会使排序器缓存失效，但不会误删算法未变化且参数兼容的 CRAFT/GraphMF 检查点。

这些恢复点优先保证结果文件一致。GraphMF、CRAFT 与 Dataset3 LightGCN 的 epoch/step 快照恢复模型权重，但没有序列化优化器动量，因此中断恢复后的最后几位浮点数可能与不中断运行不同。Dataset4 的 CPU LightGCN 是确定性逐层传播，不含优化器。

若发生显存不足，使用 `--craft-batch-size 128 --craft-inference-batch 128`，或把 `--batch-size` 从默认 32768 降到 16384；内存紧张则把 `--chunk-rows` 降到 2500，两者均不改变核心算法。

5.2 直接加载 checkpoint

该路径不重新拟合模型，只重建推理需要的紧凑数组、时序索引、稀疏传播和协同缓存：

```
1 python scripts/run_checkpoint_inference.py \  
2     --target b \  
3     --workers 8 \  
4     --chunk-rows 2500 \  
5     --cf-dim 128 \  
6     --seed 20260724
```

权重来自 `checkpoints/`。结果写入 `submissions/checkpoint_inference/`，并自动生成 `result.zip` 和 `manifest-b.json`。

若已有两个 CSV 文件，可执行打包命令：

```
1 python scripts/run_checkpoint_inference.py --package-only
```

6. 常见问题

6.1 复现注意事项

- Dataset4 会产生较大的派生缓存，建议预留至少 40GB 磁盘；
- `chunk-rows=2500` 是 18GB 内存环境的保守配置；
- Jittor、CUDA 和 LightGBM 多线程可能造成浮点末位差异，判断复现应检查输出行数、列数、有限值、范围和排序结果；

算法对应关系和规模改动见 [A榜到B榜算法改动说明](#) 与 [A B核心算法映射](#)，详细复现命令见 [B榜最优结果复现说明](#)。

6.2 Jittor 首次启动较慢

首次运行会编译 CUDA/C++ 算子，这是正常现象。请不要在编译过程中终止任务。

6.3 某阶段提示文件不存在

首先检查其上游状态文件是否存在但产物已被删除：

```
1 python3 run.py --list-stages  
2 find outputs/state -maxdepth 1 -type f | sort
```

然后强制重跑缺失产物对应的上游阶段。

7. 第三方组件

本项目通过 `requirements.txt` 使用以下主要开源组件：

- [Jittor](#)：张量计算和神经网络训练；
- [JittorGeometric](#)：图神经网络与时序图组件，安装版本固定到 requirements 中记录的提交；
- [LightGBM](#)：候选排序与树模型；
- NumPy、Pandas、SciPy、scikit-learn 和 NetworkX：数值计算、数据处理与图结构计算。

上述组件未作为源码复制进本仓库，使用时分别遵循其上游项目许可证。赛事原始数据不随源码发布，本项目也不附带第三方预训练权重。

8. 许可证

代码许可证见 [LICENSE](#)。赛事原始数据不包含在本许可证和源码包中。